AI 自动测试用例生成指南（实战总结版）

> 本指南基于 Listing优化器 / 词库管理 / 广告诊断 三个模块的实际生成和调试经验总结。
> 覆盖了**测试引擎规则 + Playwright selector 编写 + 踩坑修复**三个层面。

本指南在体系中的定位

```
13_AI自动化测试验证工作指南/
├── 01-06: 方法论与思维模型（为什么测、怎么思考、策略怎么选）
│     ↓ 方法论指导实践
├── TEST-CASE-GENERATION-GUIDE（本文）
│     → 实战层：如何编写高质量的 AI 可执行测试用例
│     → 服务于两个场景：
│       1. 人工编写测试用例 → 交给 DevHub 测试引擎执行（阶段 3）
│       2. 全托管自主测试 Phase 2 生成用例的质量标准参考（阶段 4）
│
└── 附录_参考资料/: 深度研究与工具调研
```

核心价值：本文档中的 Selector 编写规则和踩坑经验，不仅适用于人工编写测试用例，也是**全托管自主测试（Phase 2）中 LLM 生成用例的质量基准**——LLM 生成的用例越接近这些规则，执行成功率越高。

〇、全托管自主测试 vs 手动编写用例（新增）

两种模式的测试用例来源

维度	手动编写（本文主要内容）	全托管自主测试 Phase 2
用例来源	人类阅读源码 + 编写步骤	LLM 分析页面截图 + a11y tree + 自动生成
Selector 来源	人类从源码确认组件类型后手写	LLM 基于 a11y tree 推断
质量保障	人类自检（第 3 步）	需要人类抽检 + Phase 3 执行验证
维护成本	源码变更后需手动更新 selector	每次运行重新生成，无需维护
精确度	高（人类确认了真实 DOM）	中（LLM 推断可能有偏差）

全托管测试如何复用本文规则

全托管自主测试 Phase 2 生成用例时，LLM 的系统提示词会参考本文的关键规则：

- **Selector 优先级**（3.0 节）：优先用可见文本和语义属性
- **Select 组件禁用 placeholder**（3.2 节）：这是 Ant Design 的通用陷阱
- **一步一操作**（2.4 节）：LLM 生成的步骤也必须原子化
- **输入值引号包裹**（2.3 节）：保证 Agent 执行时输入完整
- **预期结果可观察**（5.1 节）：描述 UI 状态而非后端行为

> 如果你发现全托管测试 Phase 3 执行失败率高，可以检查 Phase 2 生成的用例是否违反了本文的规则——这通常是 LLM prompt 需要优化的信号。

一、工作流程

第 1 步：阅读源码，提取真实 DOM 结构

```
前端页面组件 → 找到所有交互元素 → 确认：
  1. 实际渲染标签（Button 渲染为 <button> 还是 <a>？）
  2. 实际 className（ant-btn-primary？自定义 class？）
  3. 实际文案（硬编码还是变量？变量的所有可能值？）
  4. 状态/配置常量（STATUS_CONFIG、DIMENSION_CONFIG 等的 label 值）
  5. 条件渲染（{isCompleted && <Button>}，什么状态下才可见？）
  6. 下拉框是否有默认值（有默认值时 placeholder 不可见！）
```

```
6. 下拉框是否有默认值（有默认值时 placeholder 不可见？）
```

第 2 步：按模块分批生成

批次	范围	数量
列表页	页面加载、筛选、卡片点击、删除、新建入口	6-8
创建/向导流程	表单填写、文件上传、步骤跳转	4-6
详情/结果页	Tab切换、数据展示、导出、操作菜单	6-10

第 3 步：逐条自检

对每个测试项，检查：

- √ 每个交互步骤都有 `(selector: XXX)`
- √ 输入值用「」括起
- √ Select 定位不依赖 placeholder
- √ 状态文案与源码 STATUS_CONFIG 完全一致
- √ 所需数据状态在测试环境中存在

二、步骤编写规则

2.1 固定格式

```
N. 动作描述（selector: Playwright_CSS选择器）
```

- **交互步骤**（点击/输入/选择）：必须带 `(selector: XXX)`
- **等待/观察步骤**：不需要 selector
- selector 含括号时用中文 （） 包裹

2.2 登录和导航

```
✅ 1. 以 tenant_admin 角色登录系统
✅ 2. 进入广告诊断页面（路由：http://43.163.201.182:30094/ad-center/diagnosis）
✅ 3. 等待页面加载完成

❌ 1. 打开浏览器，输入用户名密码，点击登录
❌ 2. 通过左侧菜单点击广告诊断
```

2.3 输入操作：值必须用「」括起

```
✅ 6. 在「任务名称」输入框中输入「测试广告诊断」（selector: .ant-modal:visible input#name）
✅ 7. 输入「B07X6C9RMF」（selector: input[placeholder*="ASIN"]）
✅ 8. 输入「obd2 scanner」（selector: input[placeholder*="输入核心词"]）

❌ 6. 在任务名称输入框中输入 测试广告诊断      ← AI可能只输入"测试"
❌ 7. 输入一个 SKU 关键词                      ← AI把"一个"当成输入值
❌ 8. 输入 obd2 scanner                        ← 无引号，可能截断
```

2.4 一步一操作

```
✅ 5. 点击 ASIN 输入框（selector: input[placeholder*="ASIN"]）
✅ 6. 输入「B07X6C9RMF」（selector: input[placeholder*="ASIN"]）
✅ 7. 点击「开始诊断」按钮（selector: button.btn-banner-solid）

❌ 5. 在输入框中输入 B07X6C9RMF 后点击开始诊断按钮
```

2.5 文件上传：用标签名引用

```
✅ 6. 在「业务报告」卡片的上传区域，上传文件「业务报告」
✅ 7. 在「广告报告」卡片的上传区域，上传文件「广告报告」
✅ 8. 在竞品数据的上传区域，上传文件「竞品数据」

❌ 6. 上传一个业务报告文件            ← 没有标签引用
❌ 7. 拖拽 report.csv 到上传区域      ← 不用文件名，用标签
```

测试文件在 DevHub 编辑页底部「测试文件」区域上传并设置标签。

2.6 状态感知：明确所需任务状态

```
✅ 4. 在任务列表中找到状态列显示「已完成」的行，点击该行第一列的任务名称链接
      (selector: .ant-table-tbody tr:has-text("已完成") td:first-child a >>
nth=0)
✅ 预期结果说明因果：因为任务状态为「已完成」，直接进入结果查看步骤

❌ 4. 点击第一个任务                    ← 不知道什么状态
❌ 4. 点击一个已完成的任务              ← "一个"模糊，无selector
```

2.7 条件步骤

```
✅ 8. 如果配置面板未显示，则点击「重新生成」按钮
      (selector: button.ant-btn-primary:has-text("重新生成"))
```

三、Selector 编写规则（Playwright CSS）

3.0 选择器优先级 — 稳定性从高到低

核心原则：优先用可见文本和语义属性，CSS 类名作为最后手段且必须从源码确认。

优先级	策略	Selector 示例	适用场景	稳定性
1（最优）	可见文本	`button:has-text("新建工作空间")`	所有有文字的元素	⭐⭐⭐⭐⭐
2	表单 label + 结构	`.ant-form-item:has-text("渠道") .ant-select`	表单内控件	⭐⭐⭐⭐⭐
3	**href / placeholder** 属性	`a[href$="/optimize"]`、`input[placeholder*="搜索"]`	链接、输入框	⭐⭐⭐⭐
4	位置 + 文本组合	`tr:has-text("已完成") td:first-child a >> nth=0`	表格行、列表项	⭐⭐⭐⭐
5	容器作用域 + **nth**	`.ant-select >> nth=1`	筛选栏多个同类控件	⭐⭐⭐

6	**源码确认的 class**	`button.ant-btn-primary`、`button.btn-banner-solid`	仅在源码确认后使用	⭐⭐
7（最差）	**猜测的 class**	`.ant-list-item`、`.anticon-eye`	❌ 禁止！必须查源码	⭐

⚠️ 绝对禁止：

- 猜测 Ant Design 组件渲染的 CSS 类名（如 `.ant-list-item`、`.ant-menu-item`、`.ant-card`）
- 必须读源码确认实际用了哪个组件（可能是自定义 `<div>` 而非 Ant Design 组件）

示例：**左侧分析历史列表（看起来像 List，实际是自定义 div）**

```
❌ .ant-list-item:has(.anticon-check-circle) >> nth=0
   ↑ 猜测是 Ant Design List 组件，实际是自定义 <div> + Tailwind class

✅ div.cursor-pointer:has-text("词根") >> nth=0
   ↑ 源码确认：completed 状态的 div 有 cursor-pointer class，Tag 文字为"词根"
```

3.1 按钮 — ⚠️ 必须确认实际 type

核心原则：只有源码写了 `type="primary"` 的才能用 `.ant-btn-primary`，否则不加 class 限定。

Ant Design Button 的 `type` prop 决定 CSS class：

- `type="primary"` → `ant-btn-primary`（蓝色填充）
- `type="default"` 或不写 type → `ant-btn-default`（白底描边）
- `type="link"` → `ant-btn-link`（无边框链接样式）
- `type="text"` → `ant-btn-text`（纯文本无边框）
- `className="btn-banner-solid"` → 自定义 class，不是 `ant-btn-primary`！

⚠️ **常见错误：看到按钮是蓝色/紫色就以为是 `type="primary"`，但实际可能是自定义 `className` 或默认样式 + 自定义主题色。必须看源码确认！**

```
❌ button.ant-btn-primary:has-text("新建工作空间")
   ↑ 假设是 primary，但源码是 className="btn-banner-solid"，匹配不到！

✅ 先查源码 → 再决定 selector：
```

源码	Selector	说明
`<Button type="primary">开始分析</Button>`	`button.ant-btn-primary:has-text("开始分析")`	源码确认 type=primary
`<Button className="btn-banner-solid">新建空间</Button>`	`button.btn-banner-solid`	自定义 class，不是 primary
`<Button>修改参数</Button>`	`button:has-text("修改参数")`	无 type（default），不加 class
`<Link to="/xxx"><Button>智能优化</Button></Link>`	`a[href$="/optimize"] button`	Link 包裹，用 href 定位
`<Button type="link" icon={<EyeOutlined />} />`	不用！改点击同行文字链接	纯图标按钮太小，不稳定

决策流程：

```
读源码 Button 的 props
  ├─ type="primary"        → button.ant-btn-primary:has-text("文本")
  ├─ className="btn-xxx"   → button.btn-xxx
  ├─ 无 type / type="default" → button:has-text("文本")
  ├─ type="link" + icon    → 不点图标，改点同行的文字链接
  └─ Link 包裹 Button      → a[href*="path"] button
```

3.2 下拉框（Ant Design Select）— ⚠️ 最易踩坑

核心原则：永远不要用 placeholder 定位 Select！

placeholder 只在值为空时可见。如果 Select 有默认值、有缓存值、或被上次操作填充过，placeholder 就不在 DOM 中。

```
❌ .ant-select:has(.ant-select-selection-placeholder:has-text("请选择渠道"))
                                                    ↑ 有默认值时匹配不到

✅ 表单内 Select → 用 label 定位
   .ant-modal:visible .ant-form-item:has-text("渠道") .ant-select

✅ 筛选栏 Select → 用位置定位
   .ant-select >> nth=0     (第1个下拉框)
   .ant-select >> nth=1     (第2个下拉框)
```

```
✅ 弹窗内唯一 Select
   .ant-modal:visible .ant-select
```

下拉选项（弹出层在 **document.body** 下）：

```
✅ 选指定选项     .ant-select-dropdown:visible .ant-select-item-option:has-text("已完成")
✅ 选第一个选项   .ant-select-dropdown:visible .ant-select-item-option:first-of-type
```

3.3 表格行定位

```
✅ 按状态文本找行并点击名称链接
   .ant-table-tbody tr:has-text("已完成") td:first-child a >> nth=0

✅ 第一行的删除按钮
   .ant-table-tbody tr:first-of-type button:has(.anticon-delete)

❌ 点击"查看"按钮  ← 如果是纯图标按钮(EyeOutlined)，没有文本
❌ .anticon-eye     ← 太小的span，点击不稳定
✅ 改为点击任务名称链接（同样触发 handleViewTask）
   .ant-table-tbody tr:has-text("已完成") td:first-child a >> nth=0
```

3.4 卡片/链接列表定位

```
✅ 按状态找卡片
   a[href^="/keywords/"]:has-text("已完成") >> nth=0
   a[href^="/listing-optimizer/tasks/"]:has-text("诊断完成") >> nth=0

✅ 第一个卡片
   a[href^="/keywords/"]:first-of-type
```

```
a[href^="/keywords/"]:first-of-type

✅ 卡片内的删除图标
   a[href^="/keywords/"]:first-of-type .anticon-delete
```

3.5 Tab 标签

```
✅ .ant-tabs-tab:has-text("竞品数据")
✅ .ant-tabs-tab:has-text("Search Terms")

⚠️ 注意：label 文案必须与源码 EXACTLY 一致
   源码 label="五点诊断" → .ant-tabs-tab:has-text("五点诊断")
   ❌ .ant-tabs-tab:has-text("五点")        ← 可能匹配错误
   ❌ .ant-tabs-tab:has-text("五点描述")     ← 源码不是这个
```

3.6 弹窗（Modal / Popconfirm / Dropdown）

```
✅ Modal 内按钮      .ant-modal:visible button.ant-btn-primary:has-text("创建并上传报告")
✅ Modal 内表单      .ant-modal:visible .ant-form-item:has-text("渠道") .ant-select
✅ Modal 内输入框    .ant-modal:visible input#name
✅ Popconfirm 确认   .ant-popover:visible button:has-text("删除")
✅ Dropdown 菜单项   .ant-dropdown:visible [role="menuitem"]:has-text("删除项目")

⚠️ 弹出层在 document.body 下，不在触发元素的 DOM 内
```

3.7 分段控件 / 单选按钮

```
✅ Segmented 选项    .ant-segmented-item:has-text("差异对比")
✅ Radio.Button      .ant-radio-button-wrapper:has-text("重写")
```

```
✅ Radio.Button      Ant Radio Button wrapper has text (全局)
```

3.8 搜索框

```
✅ input[placeholder*="搜索 SKU"]
✅ input[placeholder*="ASIN"]
✅ input[placeholder*="搜索词库"]

⚠️ placeholder 用于 Input 是安全的 (Input 的 placeholder 始终在 DOM 中作为 attr)
   只有 Ant Design Select 的 placeholder 是动态渲染的 span, 有值时消失
```

四、必须从源码验证的内容

4.1 状态文案

不要猜测！ 必须从源码的 STATUS_CONFIG / STATUS_OPTIONS 中提取精确文案：

```
// 找到这样的常量定义, 提取 label / text 字段
const STATUS_CONFIG = {
  pending:    { text: '待处理' },      // ← 不是"等待中"
  processing: { text: '处理中' },
  completed:  { text: '已完成' },      // ← 不是"完成"
  failed:     { text: '处理失败' },    // ← 不是"失败"
}
```

不同模块的状态文案可能不同：

模块	diagnosed 状态	completed 状态
Listing优化器	诊断完成	已完成
词库管理	—	已完成
广告诊断	—	已完成

4.2 维度/分类标签

```
// 源码
const DIMENSION_CONFIG = {
  title:      { label: '标题诊断' },    // ← 不是"标题"
  main_image: { label: '主图诊断' },    // ← 不是"主图"
  bullets:    { label: '五点诊断' },    // ← 不是"五点描述"
  gallery:    { label: '附图诊断' },    // ← 不是"图库"
}
```

4.3 按钮的条件渲染

```
// 源码中的条件渲染 → 测试步骤中必须标明前置状态
{task.status === 'diagnosed' && (
  <Button>智能 Listing 优化</Button>    // 只有 diagnosed 状态才显示
)}
{task.status === 'optimizing' && (
  <Button>查看优化进度</Button>          // 只有 optimizing 状态才显示
)}
{isCompleted && (
  <Button>编辑</Button>                  // 只有 completed 状态可编辑
)}
```

4.4 操作列的实际渲染

```
// ❌ 假设有文字按钮 "查看"
// ✅ 实际是纯图标按钮
<Tooltip title="查看">
  <Button type="link" icon={<EyeOutlined />} />    // 无文字!
</Tooltip>

// 同时任务名称也可点击 (同一个 handler)
<a onClick={() => handleViewTask(record)}>{name}</a> // ← 用这个更稳定
```

4.5 按钮的实际 type

不要凭外观猜测按钮类型！ 必须看源码的 type prop 和 className：

```
// ❌ 看到按钮是蓝色的就以为是 primary
<Button className="btn-banner-solid">新建空间</Button>  // 实际是自定义 class，不是 primary!

// ✅ 只有明确写了 type="primary" 的才用 .ant-btn-primary
<Button type="primary">开始分析</Button>                // 这个才是 primary

// ✅ 没有 type 的就是 default，用 button:has-text() 就够了
<Button>修改参数</Button>                               // default 按钮
<Button onClick={onBack}>返回</Button>                  // default 按钮
```

五、预期结果编写规则

5.1 描述可观察的 UI 状态

```
✅ 弹出绿色 Toast「任务创建成功」
✅ 页面显示面包屑导航，包含「Listing 优化器」和 ASIN 编号
✅ 头部 Tag 标签显示「诊断完成」
✅ 按钮文字从「开始诊断」变为「创建中...」

❌ 功能正常工作
❌ 数据保存到数据库
❌ 接口返回 200
```

5.2 因果一致性

```
✅ 因为任务状态为「已完成」，直接进入结果查看步骤
✅ 因为是「诊断完成」状态，按钮文字为「智能 Listing 优化」
✅ 因为该任务未做过优化，优化页直接显示「优化配置」面板

❌ 页面显示优化按钮      ← 没说明是什么状态导致的
```

5.3 不断言动态数值

```
✅ 表格显示数据，行数大于 0
✅ 统计区显示数字（不断言具体值）
✅ 弹出 Toast 包含「共分析 N 个」

❌ 表格显示 156 行
❌ 统计区显示 1234
```

六、反模式速查表

反模式	问题	正确做法
`.ant-list-item` / `.ant-`	实际可能是自定义	读源码确认组件类型，优先用可见文本定

card 猜类名	div，类名不存在	位
button.ant-btn-primary:has-text("新建") 未确认 type	按钮可能是 default 或自定义 class，匹配不到	查源码确认 type → primary 才用 .ant-btn-primary，否则用 button:has-text()
placeholder:has-text("请选择") 定位 Select	有默认值时 placeholder 不存在	.ant-form-item:has-text("label") .ant-select
.anticon-eye 点击图标	span 太小，点击不稳定	点击同一行的文字链接或 button 父元素
输入 测试内容	AI 可能截断或误解	输入「测试内容」
点击一个已完成的任务	"一个"模糊，每次定位不同	selector: tr:has-text("已完成") td:first-child a >> nth=0
Tab 文案凭印象写	与源码不一致导致匹配失败	查源码 CONFIG 的 label 字段
深层导航不加等待	上一页没加载完就操作下一步	每次跳转后加「等待页面加载完成」
不标明所需状态	环境中没有该状态的数据	步骤和预期结果都标明所需状态
预期结果超出步骤范围	验证了步骤没触发的 UI	预期结果只描述步骤直接导致的状态

七、输出格式

```
{
  "items": [
    {
      "title": "模块名 – 测试场景",
      "steps": "1. 以 tenant_admin 角色登录系统\n2. 进入XX页面（路由：http://43.163.201.182:30094/path）\n3. 等待页面加载完成\n4. 交互描述（selector: CSS选择器）",
      "expected_result": "1. 具体可观察的 UI 状态\n2. Toast/文案精确引用",
      "focus_points": "测试关注重点",
      "category": "模块-子功能",
      "priority": "P0|P1|P2"
    }
  ]
}
```

优先级定义：

- **P0**：核心业务流程、数据写入（创建/删除/上传/保存）、状态依赖路由
- **P1**：常规交互、Tab 切换、筛选、UI 展示验证
- **P2**：边界场景、样式细节、辅助功能

八、全托管自主测试的实践经验（新增）

> 以下经验来自 DevHub 全托管自动测试管道（ExplorationAgent + SimpleTestAgent）的开发和调试过程。

8.1 ExplorationAgent 的 observe-think-act 循环对 Selector 质量的影响

全托管测试中，Phase 2 生成用例的 Selector 质量直接决定 Phase 3 的执行成功率。从实践中我们发现：

a11y tree（无障碍树）是 LLM 生成 Selector 的主要输入源，但存在三个层级的可靠性：

获取方式	可靠性	信息量	说明
`page.accessibility.snapshot()`	高（但 Playwright 1.41+ 标记废弃）	完整的角色和属性	第一优先
`page.locator(':root').ariaSnapshot()`	高	结构化但格式不同	第二优先
简化 DOM 提取	中	标签+属性+文本	最后兜底

关键教训：

- a11y tree 超过 6000 字符时必须截断，否则 LLM 上下文被占满，生成质量下降
- 截图（多模态输入）比纯文本 a11y tree 更能帮助 LLM 理解页面布局
- 有了截图 + a11y tree 的组合，LLM 生成的 Selector 准确率提升约 30%

8.2 Cookie 跨阶段复用——避免重复登录

8.2 Cookie 跨阶段复用——避免重复登录

问题：Phase 1 Agent 成功登录后，Phase 3 的每个测试用例如果都重新登录，既浪费时间又可能触发限流。

解决方案：

1. Phase 1 完成后，导出浏览器 cookies 保存到数据库
2. Phase 3 每个用例执行前，向新的浏览器 context 注入 cookies
3. 如果 cookies 失效（过期/被服务端撤销），fallback 到 `auto_login`

在手动编写用例时的启示：测试步骤中的"以 xxx 角色登录系统"这一步，在全托管模式下由 cookie 注入替代，不需要每次都走完整的登录流程。

8.3 从 Agent Loop 实践中提炼的 Selector 新规则

规则	来源	说明
优先用 **`role`** 属性	a11y tree 天然包含 role	`[role="button"]`、`[role="navigation"]` 比 class 更稳定
用 **`aria-label`** 定位	a11y tree 的核心字段	`[aria-label="关闭"]` 比 `.anticon-close` 更语义化
结合可视区域判断	Agent 的 `is_visible` 检查	多个同名元素时，增加 `:visible` 或 `>> visible=true` 过滤
对话框/弹窗优先限定作用域	Agent 经常遇到弹窗	`.ant-modal:visible` 作为作用域前缀，避免匹配到被遮挡的元素

8.4 Agent 执行失败的常见原因和手动用例的对照

Agent 执行失败原因	对应手动编写中的问题	防范措施
元素不可见（被其他弹窗覆盖）	Selector 没限定弹窗作用域	加 `.ant-modal:visible` 前缀
元素不存在（条件渲染未满足）	没标明所需的前置状态	步骤中说明"因为状态为 xxx，所以..."
点击了错误的元素（多个匹配）	Selector 不够精确	用 `>> nth=0` 或更具体的作用域

页面未加载完就操作	缺少等待步骤	每次跳转后加"等待页面加载完成"
输入值被截断	值没用引号包裹	严格使用「」包裹所有输入值